

Blatt 05

5.1.1b

Das SR-Latch verhält sich je nach vorherigem Zustand unterschiedlich. Wenn $S=0$ und $R=0$ angelegt wird, speichert das Latch den vorherigen Wert, das ist das gewollte Verhalten. Das Problem entsteht wenn $S=1$ und $R=1$ gleichzeitig angelegt werden. In diesem irregulären Zustand gehen beide Ausgänge Q und $\text{not_}Q$ auf 0, was ein Widerspruch ist da sie immer entgegengesetzt sein sollten. Wenn man danach $S=0$ und $R=0$ anlegt fangen die zwei NOR-Gatter an zu oszillieren; sie können keinen stabilen Zustand finden und wechseln unendlich schnell. GHDL kann das nicht simulieren und bricht die Simulation ab, was man in GTKWave daran sieht dass die Simulation bei 30ns stoppt.

5.1.1.c

Das SR-Latch hat mehrere Nachteile. Erstens gibt es den irregulären Zustand wenn $S=1$ und $R=1$ gleichzeitig angelegt werden, beide Ausgänge gehen auf 0 was ein Widerspruch ist und zu Oszillation führt. Zweitens hat das SR-Latch keinen Takteingang, das heißt es reagiert sofort auf jede Eingabeänderung ohne dass man kontrollieren kann wann sich der Ausgang ändert. Das macht es schwierig mehrere Komponenten in einer komplexen Schaltung zu synchronisieren.

5.1.2.c

Der Vorteil des D-Latches gegenüber dem SR-Latch ist dass der irreguläre Zustand nicht mehr möglich ist. Da $S = D \text{ AND } \text{CLK}$ und $R = \text{NOT } D \text{ AND } \text{CLK}$ können S und R nie gleichzeitig 1 sein weil D und $\text{NOT } D$ immer entgegengesetzt sind. Außerdem hat das D-Latch einen Takteingang CLK der kontrolliert wann sich der Ausgang ändert, wenn $\text{CLK}=0$ ist wird der Eingangswert D ignoriert und der vorherige Zustand gespeichert.

5.2.a

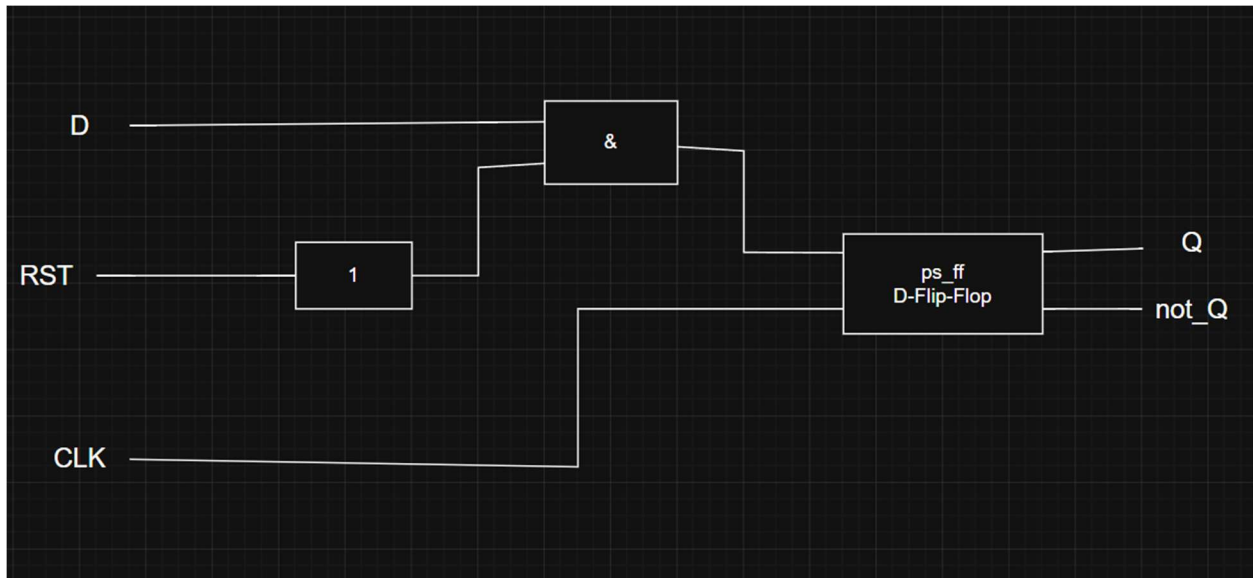
Das Primary-Secondary D-Flip-Flop besteht aus zwei hintereinandergeschalteten D-Latches. Das Primary Latch $L1$ bekommt den invertierten Takt (NOT CLK) und das Secondary Latch $L2$ bekommt den normalen Takt (CLK). Wenn $\text{CLK}=0$ ist, ist $\text{NOT CLK}=1$ also ist $L1$ offen und folgt D kontinuierlich. $L2$ ist gleichzeitig gesperrt und hält den vorherigen Wert.

Wenn $\text{CLK}=1$ wird, ist $\text{NOT CLK}=0$, also sperrt $L1$ und friert den aktuellen Wert von D ein. $L2$ öffnet sich und liest den eingefrorenen Wert von $L1$. Dadurch ändert sich Q nur genau zum Zeitpunkt der aufsteigenden Taktflanke. Die zwei Latches sind also nie gleichzeitig offen, das stellt sicher dass Q sich nur einmal pro Taktflanke ändert und nicht kontinuierlich wie beim D-Latch.

5.2.c

Der Unterschied zwischen synchronem und asynchronem Reset liegt darin wann der Reset ausgeführt wird. Beim synchronen Reset wird der Flip-Flop nur bei der aufsteigenden Taktflanke zurückgesetzt, auch wenn das Reset-Signal schon vorher aktiv ist. Beim asynchronen Reset wird der Flip-Flop sofort zurückgesetzt sobald das Reset-Signal aktiv wird unabhängig vom Takt.

5.2.d.



Für Aufgabe 5.2d und 5.2e wurde die Reset-Funktion in einer separaten Datei `ps_ff_rst.vhdl` implementiert, die das `ps_ff` als Komponente verwendet anstatt es zu kopieren.

5.3.a

Der Unterschied zwischen dem JK-Flip Flop und dem D-Flip-Flop ist dass das JK-Flip Flop zwei Eingänge J und K hat anstatt einem. Das D-Flip Flop speichert einfach den Wert von D bei der aufsteigenden Taktflanke. Das JK-Flip Flop hat vier mögliche Verhaltensweisen; Speichern, Setzen, Rücksetzen und Toggeln. Der Toggle-Fall (J=1, K=1) ist der wichtigste Unterschied, dabei wechselt Q zum entgegengesetzten Wert, was beim D-Flip Flop nicht möglich ist.

5.3.b

Die Implementierung verwendet eine behaviorale Beschreibung anstatt einer strukturellen. Das bedeutet das Verhalten wird direkt mit `if` und `elsif` Anweisungen beschrieben anstatt Gatter zu verbinden. Der `wait until rising_edge(clk)` sorgt dafür dass alle Änderungen nur bei der aufsteigenden Taktflanke passieren. Wenn `reset=1` wird q auf 0 gesetzt. Danach werden J und K überprüft. J=0 K=1 setzt q auf 0, J=1 K=0 setzt q auf 1, und J=1 K=1 toggelt q mit `not q_int`. Wenn keiner dieser Fälle zutrifft (J=0, K=0) bleibt q unverändert.